

EventVibe — Exhaustive Technical Architecture & Code Report

Complete Technical Documentation, File-by-File Breakdown, Code Logic Analysis & Viva Guide

1. Executive Summary & Application Overview

EventVibe is an end-to-end, production-ready Event Discovery and Real-Time Tracking Platform built for the Quantiphi Technical Assessment. The application integrates Ticketmaster's Discovery API for live event feeds, provides a custom React event calendar, enables single-click RSVPs and dashboard management, features a trackable friend invitation system, broadcasts live real-time updates over WebSockets, and embeds an intelligent deterministic event search assistant.

Core Architectural Principle: Strict server-side authority. All calculations (Friends Attending, click counts, upcoming event statistics, RSVP deduplication, and authorization rules) are computed exclusively on the backend. The React client acts as a purely presentation-driven SPA.

2. Complete File-by-File Breakdown & Code Logic

Below is an exhaustive documentation of every file in the codebase, detailing its contents, purpose, and key internal logic:

■ **backend/main.py — FastAPI Main Application Entry Point**

Configures FastAPI app lifespan, initializes CORS middleware, attaches exception handlers, registers routers, and exposes `/api/health` and `/ws` endpoints.

- **Lifespan Context Manager:** Calls `init_db()` on server startup to create database indexes and seed initial demo users.
 - **Global Exception Handler:** Intercepts all unhandled server exceptions, logs tracebacks to uvicorn logger, and returns JSON 500 error.
 - **CORS Middleware:** Configured for `CLIENT_URL`, `localhost:5173`, `localhost:3000` to allow smooth cross-origin browser requests.
 - **WebSocket Endpoint (`/ws`):** Registers incoming client sockets with `ws_manager`, handling connection loops and clean disconnections.

■ **backend/database.py — Database Connection & Seeding Engine**

Configures PyMongo connection to MongoDB with `serverSelectionTimeoutMS=2000` and transparent fallback to mongomock.

- **Multi-Environment Fallback:** Attempts pinging real MongoDB server; if offline/unreachable, seamlessly initializes in-memory mongomock.
 - **Index Creation:** Defines unique index on `users(email)`, unique compound indexes on `rsvps(userId, eventId)`, `invites(token)`, `invites(creatorUserId, eventId)`, and `invite_clicks(token, visitorId)`.
 - **Demo Seeding:** Checks if `users` collection is empty. If empty, inserts Demo User, Friend User, and Friend Two with default reminder settings.

■ **backend/schemas.py — Pydantic Data Schemas & Input Validators**

Defines strictly-typed Pydantic schemas for API request validation, object payloads, and model parsing.

- **ReminderSettings:** `Field(default=60, ge=1, le=10080)` enforces lead times between 1 minute and 7 days (10,080 mins).
 - **EventSnapshot:** Defines snapshot structure (`id`, `title`, `venue`, `city`, `date`, `time`, `image`, `category`) stored inside RSVPs & Invites.
 - **Request Models:** `RSVPCreateRequest`, `InviteCreateRequest`, `InviteClickRequest`, `InviteRSVPRequest`, `ReminderUpdateRequest`, `ChatRequest`.

■ **backend/websocket_manager.py — WebSocket Real-Time Manager**

Maintains active client WebSocket connection list and handles asynchronous JSON broadcasting.

- Connection Manager: Accepts connections, stores active sockets in an array, and removes disconnected sockets safely.
 - Broadcast RSVP Update: `broadcast_rsvp_update(event_id, creator_user_id, friends_attending, click_count)` packages an `rsvp_updated` payload and sends it asynchronously to all active web browser clients.

■ `backend/services/ticketmaster.py` — ***Ticketmaster API Client & Mock Fallback***

Asynchronous HTTPX client for Ticketmaster Discovery API with an automated mock event generator fallback.

- API Querying: Issues GET requests with 8.0s timeout passing city, keyword, category, date filters.
 - Normalization: Extracts nested event fields safely (venue, city, localDate, localTime, category, images), preventing external schema leakage.
 - Mock Fallback: If API key is missing or HTTP fails, returns 14 relative mock events (Music, Tech, Sports, Arts, Food across Mumbai, Pune, NY, Bangalore).

■ `backend/services/event_service.py` — ***Event Feed User State Enrichment***

Enriches raw Ticketmaster/mock event feeds with user-specific RSVP and invitation state.

- User Enrichment: Accepts `userId` and batch-queries user RSVPs and created invites.
 - State Mapping: Computes `hasRsvped` (boolean) and `friendsAttending` (integer from user's created invite) for each event card in $O(N)$ optimized time.

■ `backend/services/chat_service.py` — ***Deterministic Conversational Assistant***

Keyword and regex intent parsing engine for natural language event search and user RSVP queries.

- User RSVP Intent: Matches 'attending', 'my rsvps', or 'my events' queries, reading actual confirmed RSVPs from MongoDB for `user_id`.
 - Intent Classifier: Detects categories (Music/Concerts, Technology/AI, Sports/Matches, Arts, Food), cities (Mumbai, Pune, NY, Bangalore), and dates (today, tomorrow, this week).
 - Friendly Fallback: Returns a natural sentence response with count + event results, or helpful query suggestions if query returns 0 hits.

■ `backend/routes/events.py` — ***Event Feed Endpoint Router***

Handles GET `/api/events` query parameters (city, keyword, category, date, `userId`).

- Query Dispatch: Passes parameters to `fetch_ticketmaster_events` and enriches with user state using `enrich_events_with_user_state`.

■ `backend/routes/rsvps.py` — ***RSVP Management Router***

Handles POST `/api/rsvps`, GET `/api/rsvps/{user_id}`, DELETE `/api/rsvps/{user_id}/{event_id}`.

- POST `/api/rsvps`: Validates user, inserts RSVP document. Catches `DuplicateKeyError` -> HTTP 409.
 - GET `/api/rsvps/{user_id}`: Computes total confirmed RSVPs and upcoming count (`eventSnapshot.date >= today_str`).
 - DELETE `/api/rsvps/{user_id}/{event_id}`: Deletes RSVP. If RSVP was created via an invite token, decrements invite's `friendsAttending` (min 0) and broadcasts WebSocket update.

■ `backend/routes/invites.py` — ***Friend Invitation System Router***

Handles POST `/api/invites/{event_id}`, GET `/api/invites/{token}`, POST `/api/invites/{token}/click`, POST `/api/invites/{token}/rsvp`.

- POST `/api/invites/{event_id}`: Verifies user is RSVP'd to event. Returns existing invite or generates secure secrets token.
 - GET `/api/invites/{token}`: Fetches invite, creator name, and share link.
 - POST `/api/invites/{token}/click`: Inserts into `invite_clicks(token, visitorId)`. On new insert, increments `clickCount` by 1. Rejects creator clicks.
 - POST `/api/invites/{token}/rsvp`: Creator cannot RSVP to own invite. Inserts RSVP with `inviteToken`. On new RSVP, increments `friendsAttending` by 1 and broadcasts WebSocket update.

■ **backend/routes/users.py — User & Profile Management Router**

Handles GET /api/users, GET /api/users/{user_id}, PUT /api/users/{user_id}/reminders.

- Reminder Validation: Updates reminderSettings (enabled, beforeMinutes) with Pydantic lead time validation.

■ **backend/routes/chat.py — Chat Assistant Router**

Handles POST /api/chat endpoint.

- Query Forwarding: Forwards input query message and optional userId to chat_service.py.

■ **backend/smoke_test.py — Automated E2E Smoke Test Suite**

Executes 14 automated test assertions using FastAPI TestClient.

- Assertion Suite: Verifies health, feed, RSVPs, duplicates, dashboard, invite creation, deduplicated clicks, friend RSVPs, attendance increment/decrement, and 404 handling.

■ **frontend/src/services/api.js — Axios API & WebSocket Connection Service**

Axios instance configured with VITE_API_URL and helper functions.

- WS Derivation: getWsUrl() converts HTTP/HTTPS protocol to WS/WSS.
 - API Helpers: getEvents, createRSVP, getUserRSVPs, cancelRSVP, createInvite, getInvite, trackInviteClick, rsvpViaInvite, getUsers, getUserProfile, updateUserReminders, sendChatMessage.

■ **frontend/src/components/Navbar.jsx — Navigation Header & User Switcher**

Displays brand logo, navigation links, and demo user switcher dropdown.

- User Switcher: Fetches demo users from GET /api/users, switching active user ID in localStorage.

■ **frontend/src/components/EventCard.jsx — Responsive Event Card Component**

Displays event image, title, venue, city, date, time, category, and server-provided Friends Attending count.

- Interactive Actions: Handles Interested (RSVP) and Share (copy invite link to clipboard) actions.

■ **frontend/src/components/EventCalendar.jsx — Custom React Monthly Calendar**

Zero-dependency calendar with month navigation and weekday headers.

- Date Filter: Highlights dates with scheduled events and filters event feed on date click.

■ **frontend/src/components/RSVPCard.jsx — Dashboard RSVP Card Component**

Displays confirmed RSVP card with event snapshot details.

- Action: Supports Cancel RSVP action returning feedback toast.

■ **frontend/src/components/Chat.jsx — Collapsible Floating Event Assistant UI**

Floating drawer chat assistant interface.

- Interactive Messaging: Sends queries to POST /api/chat and displays bot text replies + inline event preview cards.

■ **frontend/src/components/Loading.jsx — Animated Loading Spinner Component**

CSS keyframe animated spinner for graceful async loading states.

- Spinner UI: Displays clean CSS keyframe spinner.

■ `frontend/src/pages/Home.jsx` — *Home Discovery Feed & Real-Time Listener*

Search bar, city dropdown, category dropdown, calendar, and event grid.

- **WebSocket Sync:** Listens to `rsvp_updated` events on `/ws` to update card attendance live.

■ `frontend/src/pages/Dashboard.jsx` — *RSVP Dashboard Page*

Displays Total Confirmed RSVPs and Upcoming Events stat counters.

- **Data Source:** Renders `RSVPCards` based on server responses.

■ `frontend/src/pages/Profile.jsx` — *User Profile & Notification Settings Page*

Displays account details and reminder settings form.

- **Reminder Form:** Enabled checkbox + lead time in minutes (1–10,080 mins).

■ `frontend/src/pages/Invite.jsx` — *Friend Invitation Page (/invite/:token)*

Displays inviter name, event snapshot, `clickCount`, `friendsAttending`.

- **Visitor Flow:** Registers visitor click once on load, handles I'm Interested (RSVP) action.

■ `frontend/src/App.jsx` — *Main React App Component & Router*

Configures React Router routes (`/`, `/dashboard`, `/profile`, `/invite/:token`).

- **App Container:** Manages active user state, global Navbar, Chat, and Toast notifications.

■ `frontend/src/main.jsx` — *React DOM Entry Point*

Mounts App component inside `React.StrictMode`.

- **Entry Point:** Mounts into HTML `#root` element.

■ `frontend/src/index.css` — *Global Dark Design System CSS*

CSS design system and tokens.

- **Styling:** Glassmorphism backdrop filters, HSL color palette, button utilities, card grids, calendar styles, and chat animations.

3. Core Engineering Mechanics & Business Rules

- **Atomic Unique-Index Deduplication:** Duplicate operations (e.g. double RSVPs, repeat friend RSVPs, or duplicate visitor clicks) are prevented directly by MongoDB database unique indexes. Application code catches `DuplicateKeyError` and converts it into HTTP 409 Conflict without risking race conditions.
- **Immutable Event Snapshots:** When an RSVP or invite is created, a full snapshot of the event metadata (title, venue, city, date, time, image, category) is frozen inside the document. This guarantees that user dashboards remain functional even if external Ticketmaster data changes.
- **Click Tracking vs. Friends Attending:** The platform strictly separates curiosity (clicks) from commitment (attendance). `clickCount` tracks unique link visits via `invite_clicks(token, visitorId)`. `friendsAttending` tracks confirmed friend RSVPs created via the invite. Clicks NEVER inflate attendance numbers.
- **Real-Time WebSocket Architecture:** When an RSVP occurs via an invite or is cancelled, FastAPI's `websocket_manager` broadcasts a JSON payload containing `{ type: 'rsvp_updated', eventId, creatorUserId, friendsAttending, clickCount }`. Connected React clients parse this and update affected card states in real-time.
- **Ticketmaster API Isolation & Fail-Safe Fallback:** Ticketmaster API calls occur strictly on the backend inside `ticketmaster.py` with an 8-second timeout. If `TICKETMASTER_API_KEY` is missing or the request fails, the application automatically switches to 14 structured mock events across cities and categories.

4. Automated Verification & Test Results

The application was thoroughly validated using an automated end-to-end smoke test script (backend/smoke_test.py). All 14 assertions passed cleanly:

- [PASSED]** 1. /api/health responds with HTTP 200 OK.
- [PASSED]** 2 & 3. /api/events returns normalized feed (source=mock/ticketmaster).
- [PASSED]** 4. Demo User 1 can RSVP to an event.
- [PASSED]** 5. Duplicate RSVP correctly rejected with HTTP 409 Conflict.
- [PASSED]** 6. Dashboard returns user RSVPs with accurate total & upcoming stats.
- [PASSED]** 7. User 1 successfully creates shareable invite link.
- [PASSED]** 8. Invite GET returns valid inviter name and event snapshot.
- [PASSED]** 9. Visitor click tracking deduplicates repeat visits from same visitorId.
- [PASSED]** 10 & 11. Friend User 2 RSVPs via invite and friendsAttending increments to 1.
- [PASSED]** 12. Duplicate friend RSVP rejected with 409 without inflating attendance.
- [PASSED]** 13. Cancellation of friend RSVP decrements friendsAttending back to 0.
- [PASSED]** 14. Invalid invite token returns HTTP 404 Not Found.

Frontend Build Verification: Ran npm run build (vite build) — 100 modules transformed, 0 errors.

5. Viva Masterclass & Examination Guide

10-Line Architecture Summary for Viva Presentation:

1. React + Vite SPA renders UI components and handles client routing.
2. Axios sends REST HTTP calls to FastAPI server endpoints on Uvicorn.
3. FastAPI enforces input validation using Pydantic schemas.
4. Ticketmaster Discovery API queried on backend via HTTPX (8s timeout).
5. Automatic fallback to mock events if API key is missing or request fails.
6. MongoDB stores users, rsvps, invites, and invite_clicks collections.
7. MongoDB unique compound indexes prevent duplicate RSVPs and clicks.
8. Real-time updates broadcast over WebSockets (/ws) on RSVP actions.
9. React WebSocket client listens and updates affected cards live.
10. Deterministic chat assistant parses natural queries via regex matching.

Top 15 Likely Viva Questions & Answers:

Q1: Why React + Vite for the frontend?

React provides declarative component rendering perfect for updating cards and calendar states. Vite provides instant HMR and fast production builds.

Q2: Why FastAPI for backend?

FastAPI offers high speed, automatic Pydantic validation, native WebSocket support, and auto-generated Swagger documentation.

Q3: Why MongoDB?

MongoDB's flexible document structure naturally fits nested event snapshots and user reminder settings without requiring complex relational joins.

Q4: Why keep Ticketmaster API on backend?

Keeps API keys secret, prevents CORS issues, normalizes raw responses, and handles fallback to mock data transparently.

Q5: How are duplicate RSVPs prevented?

By creating a unique compound index on (userId, eventId) in MongoDB. Attempting a duplicate raises DuplicateKeyError, returning HTTP 409 Conflict.

Q6: Difference between clickCount and friendsAttending?

clickCount tracks unique link visitors via (token, visitorId) compound index. friendsAttending counts actual confirmed RSVPs created through the invite.

Q7: How do real-time updates work?

When an RSVP occurs, websocket_manager.py broadcasts an rsvp_updated JSON payload to all /ws clients. React updates the affected card state dynamically.

Q8: How does the chat assistant work?

chat_service.py uses regex pattern matching to parse categories, cities, dates, and personal user RSVP records against MongoDB.

Q9: Why store eventSnapshots in RSVPs?

External API data can change or expire. Storing an immutable snapshot guarantees the user's dashboard displays consistent data.

Q10: Why is PyMongo synchronous in FastAPI?

PyMongo is simple and reliable for MVPs, executing in <1ms for local/in-memory database operations.

Q11: How is the Ticketmaster API key kept safe?

Stored in server environment variables (backend/.env), ignored in .gitignore, and never sent to React.

Q12: How does the calendar component work?

Custom React calendar calculating days in month dynamically, highlighting dates with events, and filtering feeds on date click.

Q13: How does the user profile reminder validation work?

Pydantic Field(ge=1, le=10080) validates lead times between 1 minute and 7 days on the backend.

Q14: What happens if MongoDB server is offline?

backend/database.py catches connection failure and transparently falls back to an in-memory mongomock database.

Q15: What would you change for a production deployment?

Add JWT/OAuth authentication with bcrypt, upgrade to Motor for async MongoDB IO, and deploy MongoDB Atlas.